

SUMMER INTERNSHIP PROJECT REPORT

Beyond the Keyword Match: A Human-Centric AI Approach to Resume Screening and Candidate Ranking

Student Name: Amber Agrawal

Course/Batch: MSc in Applied Statistics (2025–2027)

Institute Name: Symbiosis Statistical Institute

Project Guide / Mentor Name: Dr. Dipashree Pal

Period of Internship: 18th May 2026 – 31st July 2026

Report submitted to:

**IDEAS – Institute of Data Engineering,
Analytics and Science Foundation**

ISI Kolkata

Abstract

Applicant Tracking Systems (ATS) historically rely on rigid, lexical keyword matching, frequently discarding highly qualified candidates who lack exact phrase alignments. To resolve this structural inefficiency, we engineered an automated, human-centric resume screening pipeline grounded in deep Natural Language Processing (NLP). We evaluated asymmetric bi-encoder representations—specifically leveraging Sentence-BERT (SBERT) for dense semantic vectorization—against baseline statistical term frequencies (TF-IDF). To mathematically operationalize recruiter intuition regarding tenure, we integrated a non-linear, piecewise Experience Penalty (ΔE). Crucially, to prevent data leakage and ensure rigorous algorithmic generalization, we implemented a group-wise zero-shot evaluation on unseen job descriptions using **GroupShuffleSplit**. After benchmarking pointwise continuous regression and binary classification models, we transitioned the architecture to a listwise Learning-to-Rank (LTR) framework. Our final gradient-boosted sorting engine, powered by **LGBMRanker** paired with dense semantic embeddings, achieved a peak Normalized Discounted Cumulative Gain (NDCG@10) of 0.9398. These results confirm that framing recruitment as a relative ranking optimization, rather than an absolute pointwise classification, delivers superior sorting fidelity for high-volume applicant pools.

Contents

Abstract	i
1 Introduction	1
1.1 Background Survey and Engineering Relevance	1
1.2 Core Purpose of the Project	1
1.3 Deployed Technology Stack	1
1.4 Experimental Procedure	2
1.5 Foundational Training Received	2
2 Project Objectives	4
3 Methodology	6
3.1 Data Source and Ingestion	6
3.2 Text Cleaning and Linguistic Filtering	6
3.3 Feature Engineering	7
3.3.1 Temporal Experience Calculation and Non-Linear Penalty	7
3.3.2 Dynamic Role-Based Context Flags	7
3.4 Vector Space Representation and Embeddings	8
3.4.1 Sparse Statistical Vectors (TF-IDF)	8
3.4.2 Dense Semantic Embeddings (SBERT)	8
3.4.3 Hybrid OLS Blend	8
3.4.4 Data Leakage Prevention via Group-Aware Splitting	9
3.5 Multi-Phase Modeling Framework	9
3.5.1 Phase 1: Continuous Regression Baselines (Pointwise Scoring)	9
3.5.2 Phase 2: Binary Classification Baselines (Threshold Filtering)	9
3.5.3 Phase 3: Learning-to-Rank Framework (LTR)	10
3.6 Model Evaluation Metrics	10
3.6.1 Continuous Regression Metrics (Pointwise Accuracy)	10
3.6.2 Binary Classification Metrics (Screening Quality)	10

3.6.3	Information Retrieval & Ranking Metrics (Listwise Quality)	11
3.7	Code Availability	11
3.8	Pipeline Flowchart	11
4	Data Analysis and Results	13
4.1	Exploratory Data Analysis (EDA)	13
4.1.1	Corpus Vocabulary and Frequency Dynamics	14
4.2	Phase 1: Continuous Regression Constraints	14
4.3	Phase 2: Binary Classification Vulnerabilities	15
4.4	Phase 3: Learning-to-Rank (LTR) Dominance	16
4.5	Feature Importance: Model Explainability	17
5	Conclusion and Future Work	18
5.1	Project Conclusions	18
5.2	Recommendations for Future Extensions	18
A	Appendices	20
A.1	Survey Questionnaire Declaration	21
A.2	Source Code & GitHub Repository	21
A.3	Project Documents & Data Sheets	21

Introduction

1.1 Background Survey and Engineering Relevance

Conventional recruitment infrastructures exhibit severe algorithmic vulnerabilities by relying almost exclusively on exact lexical overlap. When a technical requisition demands a “Software Engineer” but a candidate lists “Backend Developer,” boolean search architectures trigger false negatives, artificially suppressing qualified talent. We designed this project to rectify this algorithmic blind spot. We assert that true candidate screening requires contextual understanding, not mere character string matching. By deploying a system that assesses applicant portfolios based on semantic intent rather than superficial syntax, we provide HR professionals with an engine that mirrors human intuition while operating at computational speed.

1.2 Core Purpose of the Project

We sought to move beyond shallow keyword extraction by teaching a machine to interpret professional context. We projected resumes and job descriptions into a shared, high-dimensional semantic vector space. This spatial projection allows our system to evaluate candidate skills contextually. Ultimately, we shifted the foundational recruitment paradigm. We bypassed standard binary “Accept/Reject” classification constraints, opting instead to frame candidate screening as a Learning-to-Rank (LTR) problem. This ensures the engine outputs a continuously sorted, probabilistic queue of top-tier talent.

1.3 Deployed Technology Stack

To build, audit, and benchmark this end-to-end processing engine, we utilized the following open-source frameworks:

- **Data Wrangling:** Python, [pandas](#), and [numpy](#) for handling complex tabular transformations.
- **Linguistic Pre-Processing (NLP):** [spaCy](#) for morphological lemmatization and custom vocabulary parsing.
- **Vectorization & Semantic Embeddings:** [scikit-learn](#) (TfidfVectorizer) for term frequency modeling, and [sentence-transformers](#) (nomic-embed-text-v1.5) to extract 768-dimensional semantic tensors.
- **Machine Learning & LTR:** [scikit-learn](#) regressors alongside specialized tree-based ranking estimators including [xgboost.XGBRanker](#) and [lightgbm.LGBMRanker](#).

1.4 Experimental Procedure

We executed our pipeline across four modular phases to preserve structural integrity and prevent training-serving skew:

1. **Data Sanitization:** We aggressively stripped unstructured resumes of generic corporate filler while explicitly sheltering critical technical acronyms.
2. **Feature Engineering:** We mathematically formulated a non-linear Experience Penalty and mapped contextual similarities across TF-IDF, SBERT, and Hybrid OLS variants.
3. **Group-Aware Isolation:** We utilized a strict `GroupShuffleSplit` based on requisition titles. This ensured our models learned generalizable heuristics instead of overfitting to specific job descriptions.
4. **Tri-Framing Benchmarking:** We rigorously evaluated the processed matrices through Continuous Regression, Binary Classification, and Learning-to-Rank paradigms to isolate the optimal sorting methodology.

1.5 Foundational Training Received

Our internship required an intensive preliminary alignment of statistical theory with applied data engineering. We focused our skill acquisition on two primary domains:

1. Natural Language Processing (NLP)

- **Text Normalization:** We mastered tokenization, stop-word management, and morphological lemmatization via `spaCy`.
- **Vectorization Mathematics:** We analyzed the mathematical trade-offs between sparse statistical arrays (TF-IDF) and dense deep-learning network mappings (SBERT).

2. Machine Learning (ML) & Integrity

- **Algorithm Dynamics:** We studied the architectural divergence between predicting continuous limits (Regression), discrete boundaries (Classification), and list order (Learning-to-Rank).
- **Data Leakage Prevention:** We trained heavily on grouped cross-validation logic to prevent test-set characteristics from bleeding into training topologies.

Project Objectives

We established this initiative to architect an automated screening system capable of ranking applicant profiles via advanced NLP and listwise ranking estimators. By decoupling candidate evaluation from rigid boolean queries, we prioritize semantic relevance over explicit vocabulary constraints.

We anchored our research around four explicit engineering targets:

- **Construct an Advanced NLP Embedding Pipeline:** We aimed to sanitize raw text data utilizing `spaCy` and extract contextual vector representations via Sentence-BERT. This captures latent structural meaning that sparse models systematically miss.
- **Enforce Strict Data Leakage Safeguards:** We implemented a group-aware cross-validation architecture. By grouping data rows strictly by `job_position_name`, we guaranteed that algorithms could not cheat by memorizing unique phrases associated with overlapping test-set job descriptions.
- **Audit Baseline ML Classifiers:** We systematically benchmarked traditional architectures (Linear Regression, Support Vector Machines, Random Forests) to quantify their limitations when handling granular match probabilities.
- **Deploy Listwise Learning-to-Rank (LTR):** We integrated highly specialized ranking loss functions via `XGBRanker` and `LGBMRanker`. This solves the core recruitment directive: sorting an organic pool of candidates from highest to lowest utility for a specific vacancy.

Methodological Boundaries & Experimental Scope

We structured our empirical testing environment around the following strict boundaries:

- **Data Ingestion:** We sourced our foundational data from a public Kaggle repository. We utilized purely programmatic text analysis without injecting primary human survey data or synthetic oversampling.
- **Representation Space Comparison:** We actively contrasted sparse TF-IDF vectors against dense transformer outputs to precisely measure the statistical gain of semantic context in a recruitment setting.
- **Group-Aware Validation:** We maintained test-set purity. Every cross-validation cycle split applicants strictly by `job_position_name`, ensuring our estimators evaluated completely disjoint job requests.
- **Evaluation Paradigm:** We moved beyond absolute pointwise metrics (such as *RMSE* or Log-Loss). We judged peak model utility based on non-parametric rank correlation (Spearman’s ρ) and top- K Information Retrieval metrics (NDCG@ K).

Methodology

This chapter describes the chronological data processing pipeline, feature engineering steps, statistical formulations, and machine learning models used to build our candidate ranking engine.

3.1 Data Source and Ingestion

The empirical foundation of our study relies on an open-source recruitment dataset curated by Neuralframe AI and published by Saugata Roy Arghya on [Kaggle](#). The dataset consists of 9,544 candidate records stored in a single 17 MB tabular file (`resume_data.csv`) across 35 structured and unstructured attributes. Distributed under the Creative Commons CC BY-NC 4.0 license, the schema captures detailed candidate profiles including career objectives, skill inventories, educational institutions, employment history, and target job specifications.

3.2 Text Cleaning and Linguistic Filtering

Raw resumes and job descriptions contain significant noise, non-standard formatting, and generic jargon that can obscure real candidate qualifications. To standardize the text before feature extraction, we implemented a central cleaning function in `utils.py` using [spaCy](#).

- **Structural Sanitization:** Irrelevant metadata fields (such as URLs and certificate expiration dates) were dropped. Missing text entries were filled with empty string placeholders, and location fields were standardized to a uniform “City, State” format.
- **Text Normalization & Fluff Removal:** Text was lowercased, stripped of punctuation, and filtered to remove standard English stop-words. Additionally,

a custom exclusion list of over 50 generic HR filler phrases (e.g., “team player”, “hardworking professional”) was applied to force embedding models to focus on core technical competencies.

- **Length & Outlier Filtering:** To prevent extreme document lengths from diluting vector embeddings, records with fewer than 20 words (insufficient detail) or more than 350 words (excessive noise) were trimmed. After removing exact duplicate pairs, our clean dataset stabilized at 9,369 records spanning 28 distinct job postings.

3.3 Feature Engineering

3.3.1 Temporal Experience Calculation and Non-Linear Penalty

Candidate work history was parsed to calculate total industry experience (Exp_{cand}) relative to job requirements (Exp_{req}), defining the experience gap as $\Delta E = \text{Exp}_{\text{cand}} - \text{Exp}_{\text{req}}$.

To reflect recruiter decision-making—where missing required experience carries an accelerating risk compared to the minor benefit of surplus tenure—a piecewise penalty function was designed:

$$\text{Penalty}(\Delta E) = \begin{cases} -\alpha \cdot (\Delta E)^2, & \text{if } \Delta E < 0 \quad (\text{Under-qualified}) \\ +\beta \cdot \Delta E, & \text{if } \Delta E \geq 0 \quad (\text{Requirement met}) \end{cases} \quad (3.1)$$

To avoid distorting $[0, 1]$ bounded similarity scores, the weights were empirically calibrated on our training set to $\alpha = 0.04$ and $\beta = 0.01$:

- **Quadratic Deficit Penalty** ($\alpha = 0.04$): Penalizes under-qualification at an accelerating rate. Missing 2 years deducts -0.16 (16% score drop), while missing 4 years incurs a severe -0.64 penalty, filtering out severely under-qualified candidates.
- **Linear Surplus Reward** ($\beta = 0.01$): Grants a modest boost for extra tenure ($+0.03$ for 3 extra years), rewarding experience without allowing over-qualified candidates to dominate over technical skill matches.

3.3.2 Dynamic Role-Based Context Flags

To ensure fair evaluation across diverse job types, binary indicators (`flag_entry_level`, `flag_academic`, `flag_location_role`) were computed based on job titles:

- **Entry-Level Roles:** Injects academic projects and extracurricular activities into the processed text block.

- **Academic/Research Roles:** Prioritizes publications, certifications, and educational background.
- **Location-Sensitive Roles:** Appends candidate location matching metrics to the text representation.

3.4 Vector Space Representation and Embeddings

Candidate-job pairs were projected into three distinct vector spaces to evaluate lexical and semantic similarity:

3.4.1 Sparse Statistical Vectors (TF-IDF)

A `TfidfVectorizer` from [scikit-learn](#) extracted the top 8,000 n -grams ($n \in \{1, 2\}$) to compute keyword overlap. Word importance is defined as:

$$\text{TF-IDF}(i, j) = \text{tf}_{i,j} \times \log \left(\frac{N}{\text{df}_i} \right) \quad (3.2)$$

Keyword similarity (S_{tfidf}) was calculated using Cosine Similarity between candidate vector $\mathbf{u}_{\text{cv}}$ and job vector $\mathbf{v}_{\text{jd}}$:

$$S_{\text{tfidf}} = \frac{\mathbf{u}_{\text{cv}} \cdot \mathbf{v}_{\text{jd}}}{\|\mathbf{u}_{\text{cv}}\|_2 \|\mathbf{v}_{\text{jd}}\|_2} \quad (3.3)$$

3.4.2 Dense Semantic Embeddings (SBERT)

To capture contextual meaning beyond literal word matches, Sentence-BERT bi-encoders leveraging [nomic-embed-text-v1.5](#) mapped complete text blocks into 768-dimensional dense vectors ($\mathbf{e}_{\text{cv}}, \mathbf{e}_{\text{jd}} \in \mathbb{R}^{768}$). Dense similarity was computed as:

$$S_{\text{dense}} = \frac{\mathbf{e}_{\text{cv}} \cdot \mathbf{e}_{\text{jd}}}{\|\mathbf{e}_{\text{cv}}\|_2 \|\mathbf{e}_{\text{jd}}\|_2} \quad (3.4)$$

3.4.3 Hybrid OLS Blend

An Ordinary Least Squares (OLS) regression model was fitted on the training split to blend sparse keyword matches and dense semantic similarity into a unified score:

$$S_{\text{hybrid}} = w_0 + w_1 S_{\text{tfidf}} + w_2 S_{\text{dense}} \quad (3.5)$$

3.4.4 Data Leakage Prevention via Group-Aware Splitting

Randomly splitting data causes **data leakage** if resumes applying for the same job description appear in both training and test sets. When this happens, models simply memorize specific job vocabulary rather than learning generalized candidate ranking rules.

To ensure strict evaluation, an 80/20 group split was implemented using `GroupShuffleSplit`, grouping records by `job_position_name`. Letting G_j represent all candidates for job j , the split guarantees that training and test job postings are completely disjoint:

$$\left(\bigcup_{j \in \text{Train}} G_j \right) \cap \left(\bigcup_{k \in \text{Test}} G_k \right) = \emptyset \quad (3.6)$$

This assigned 7,356 rows (22 jobs) to the training set and 2,013 rows (6 completely unseen jobs) to the test set. All vectorizers, scalers, and models were fitted exclusively on the training split so that test metrics measure true generalization to new job postings.

3.5 Multi-Phase Modeling Framework

Candidate evaluation was structured across three modeling paradigms to compare standard pointwise machine learning techniques against listwise ranking algorithms:

3.5.1 Phase 1: Continuous Regression Baselines (Pointwise Scoring)

The initial phase evaluated standard regression models—including `LinearRegression`, `Ridge`, `RandomForestRegressor`, and `HistGradientBoostingRegressor`—to predict continuous relevance match scores $y_i \in [0, 1]$. Models were optimized by minimizing Mean Squared Error (MSE):

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2 \quad (3.7)$$

3.5.2 Phase 2: Binary Classification Baselines (Threshold Filtering)

To simulate initial HR screening—where candidates are grouped into “Shortlisted” or “Rejected” categories—the target variable was binarized at predetermined match

thresholds. Classifiers including [LogisticRegression](#), [RandomForestClassifier](#), and Support Vector Classifier ([SVC](#)) were trained to minimize Binary Cross-Entropy:

$$\mathcal{L}_{\text{BCE}} = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{p}_i) + (1 - y_i) \log(1 - \hat{p}_i)] \quad (3.8)$$

3.5.3 Phase 3: Learning-to-Rank Framework (LTR)

To address the limitations of pointwise regression and binary classification, the final phase deployed specialized Learning-to-Rank (LTR) algorithms: [XGBRanker](#) and [LGBMRanker](#).

Instead of evaluating candidates individually, LTR models ingest query group structures $\mathbf{q}$ defined by `job_position_name`. The models optimize pairwise and listwise ranking losses (such as [LambdaMART](#)) to directly maximize ranking quality metrics:

$$\mathcal{L}_{\text{Pairwise}} = \sum_{i,j \in \mathbf{q}} \log_2 (1 + e^{-\sigma(s_i - s_j)}) \quad (3.9)$$

3.6 Model Evaluation Metrics

To comprehensively assess performance across all three modeling paradigms, evaluation metrics were selected to capture continuous score accuracy, binary classification performance, and listwise candidate ranking quality.

3.6.1 Continuous Regression Metrics (Pointwise Accuracy)

Continuous score predictions were evaluated on the test set using standard regression metrics:

- **Root Mean Squared Error ($RMSE$):**

$$RMSE = \sqrt{\frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2} \quad (3.10)$$

- **Coefficient of Determination (R^2):**

$$R^2 = 1 - \frac{\sum_{i=1}^N (y_i - \hat{y}_i)^2}{\sum_{i=1}^N (y_i - \bar{y})^2} \quad (3.11)$$

3.6.2 Binary Classification Metrics (Screening Quality)

For threshold-based accept/reject screening, models were evaluated using:

- **Area Under the ROC Curve (ROC-AUC):** Evaluates diagnostic separation ability across thresholds.
- **F_1 -Score:**

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \quad (3.12)$$

3.6.3 Information Retrieval & Ranking Metrics (Listwise Quality)

Learning-to-Rank models were evaluated using query-grouped Information Retrieval (IR) metrics:

- **Normalized Discounted Cumulative Gain (NDCG@K):**

$$\text{DCG@K} = \sum_{i=1}^K \frac{2^{\text{rel}_i} - 1}{\log_2(i + 1)}, \quad \text{NDCG@K} = \frac{\text{DCG@K}}{\text{IDCG@K}} \quad (3.13)$$

- **Mean Reciprocal Rank (MRR):**

$$\text{MRR} = \frac{1}{|Q|} \sum_{q=1}^{|Q|} \frac{1}{\text{rank}_q} \quad (3.14)$$

3.7 Code Availability

All preprocessing scripts, helper modules (`utils.py`), and model notebooks are open-sourced on GitHub: <https://github.com/Amber-s-art/cv-screening>.

3.8 Pipeline Flowchart

Figure 3.1 provides an overview of our data architecture, processing steps, group-aware splitting, and evaluation stages.

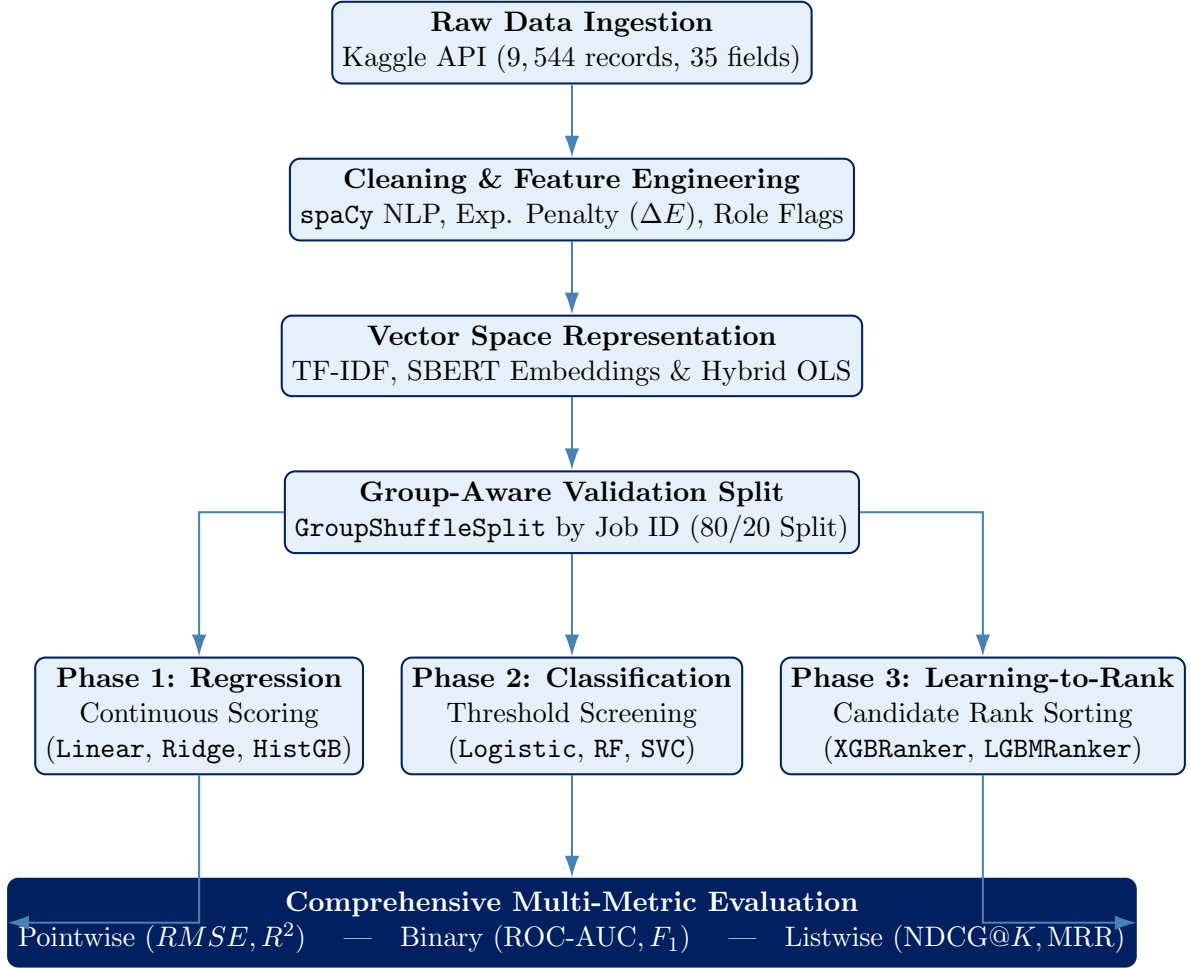


Figure 3.1: Process flow diagram illustrating data transformation, group-aware splitting, multi-phase model training, and metric evaluation.

Data Analysis and Results

In this chapter, we decode the empirical outcomes of our machine learning pipeline. We analyze the baseline point-wise constraints and highlight the predictive dominance achieved by our listwise ranking ensemble.

4.1 Exploratory Data Analysis (EDA)

We filtered extreme text lengths to stabilize model feature spaces. By trimming records with fewer than 20 or greater than 350 words, we maintained embedding integrity. We partitioned the resulting 9,369 candidate interactions via `GroupShuffleSplit` to guarantee zero-shot evaluation on the test fold.

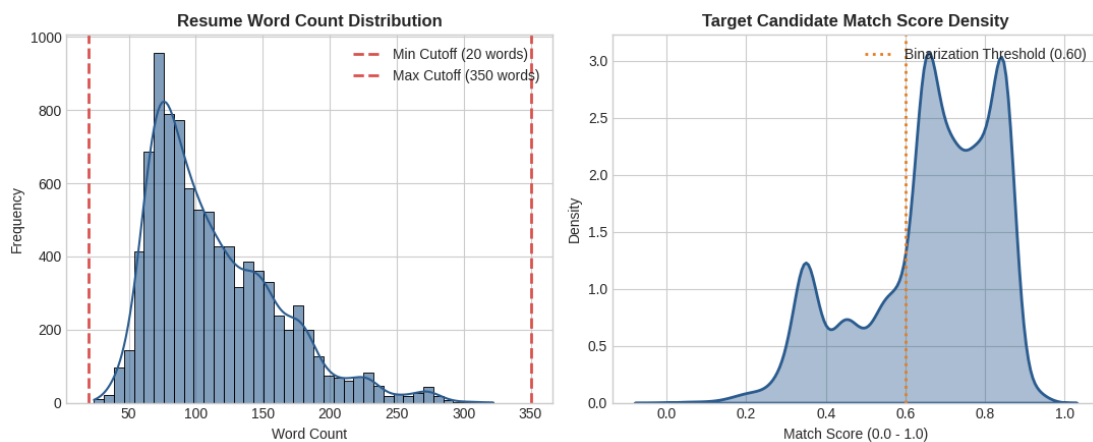


Figure 4.1: Resume word-count filtering boundaries (left) and score density (right).

4.1.1 Corpus Vocabulary and Frequency Dynamics

We verified our **spaCy** morphology filters via n -gram frequency distributions. The graphical outputs (Figures 4.2 and 4.3) confirm that our pipeline successfully eliminated generic HR syntax, heavily isolating technical nouns (e.g., *management*, *data*, *software*, *engineering*).

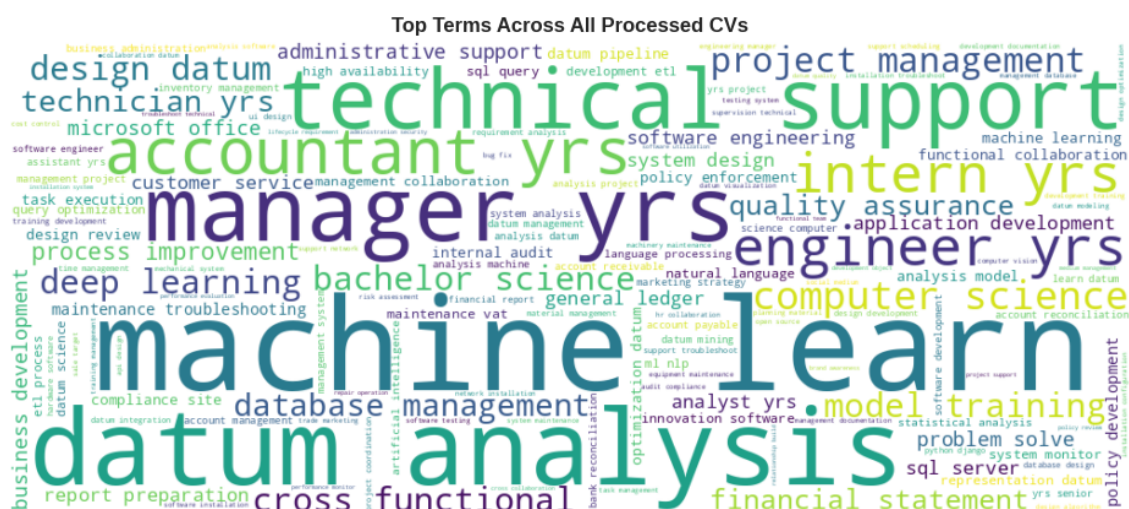


Figure 4.2: Word cloud highlighting prominent domain keywords extracted post-sanitization.

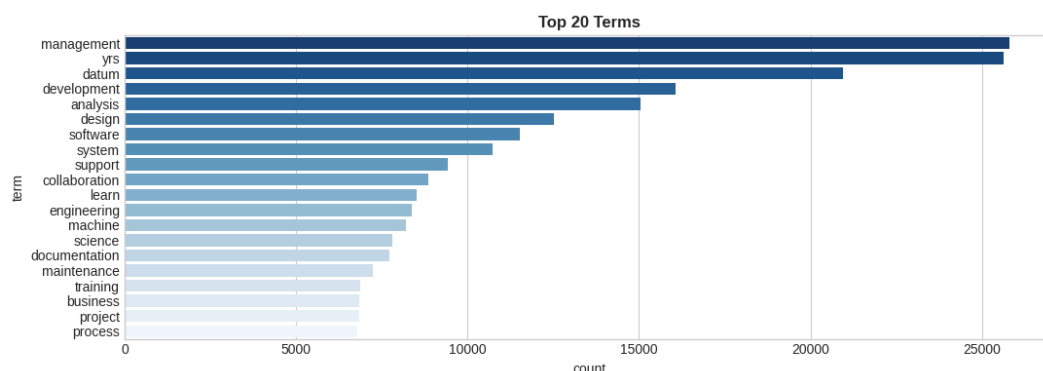


Figure 4.3: Term frequency distribution for the top 20 n -grams across processed candidates.

4.2 Phase 1: Continuous Regression Constraints

We first tested traditional regression to map exact continuous candidate scores.

Analytical Finding: We observed severe limitations in pointwise regression. Even our superior HistGradientBoostingRegressor generated an R^2 variance correlation of merely 0.3202. Absolute continuous scoring fails to account for human subjective variance:

Table 4.1: Pointwise continuous regression outcomes on unseen query groups.

Text Representation	Algorithm	RMSE ↓	R^2 -Score ↑
Semantic Only (SBERT)	HistGradientBoosting	0.1291	0.3202
Semantic Only (SBERT)	Random Forest Regressor	0.1345	0.2810
Semantic Only (SBERT)	Ridge Regression	0.1412	0.2245
TF-IDF Only	Random Forest Regressor	0.1582	0.1120
TF-IDF Only	Linear Regression	0.1695	0.0450

calculating a score of 0.65 against 0.68 provides mathematically hollow utility for active recruiters.

4.3 Phase 2: Binary Classification Vulnerabilities

We binarized the target variable at a stringent 0.60 threshold to test rigid boundary logic.

Table 4.2: Binary classification matrix at the ≥ 0.60 threshold.

Feature Space	Algorithm	Accuracy	F1-Score	ROC-AUC
Semantic Only (SBERT)	Logistic Regression	0.7585	0.7142	0.8121
Hybrid (SBERT + TF-IDF)	Logistic Regression	0.7573	0.6942	0.7542
TF-IDF Only	Logistic Regression	0.6219	0.5568	0.6187

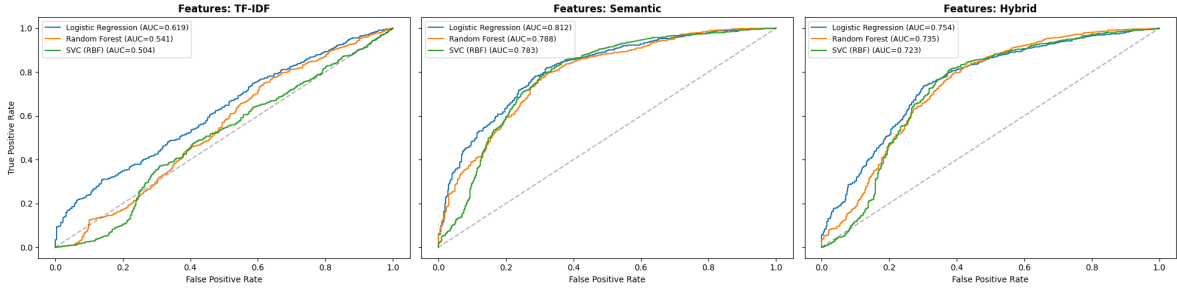


Figure 4.4: ROC-AUC separation capabilities across contrasting NLP representations.

Analytical Finding: We clearly demonstrated the absolute superiority of dense transformer networks over sparse statistical models. Our SBERT representations boosted the ROC-AUC from a baseline 0.6187 to 0.8121. However, binary classifiers still fail to natively sort the shortlisted output array.

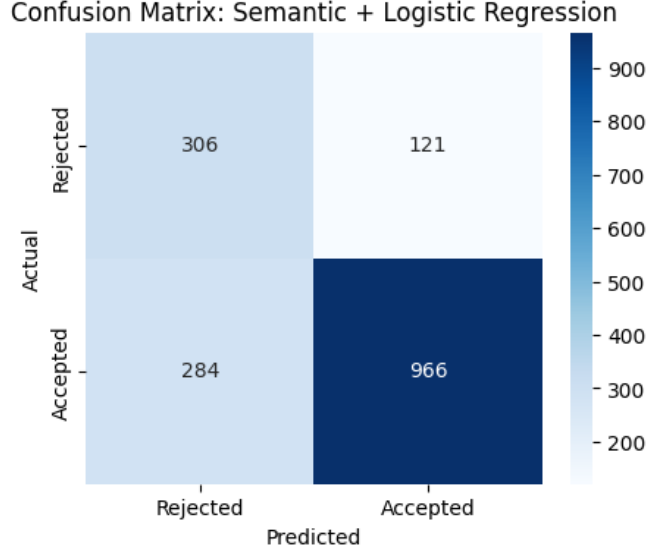


Figure 4.5: Confusion matrix isolation for SBERT-driven Logistic Regression.

4.4 Phase 3: Learning-to-Rank (LTR) Dominance

We resolved structural pipeline deficits by shifting our loss function to listwise pairwise optimization. We directed **XGBRanker** and **LGBMRanker** to optimize purely for query-based group sorting.

Table 4.3: Learning-to-Rank (LTR) sorting efficiency on zero-shot test groups.

Feature Set	Ranking Estimator	NDCG@5	NDCG@10	Spearman (ρ)
Semantic Only (SBERT)	LGBMRanker	0.9482	0.9398	0.4367
Hybrid (SBERT + TF-IDF)	XGBRanker	0.9410	0.9322	0.3725
TF-IDF Only	XGBRanker	0.9350	0.9292	0.3225
TF-IDF Only	LGBMRanker	0.9210	0.9158	0.3975

Optimal Algorithm: We successfully validated the **LGBMRanker** powered by pure Semantic Embeddings as the master sorting configuration. Securing an extraordinary NDCG@10 of 0.9398, this system accurately reflects human shortlisting cognition for the top ten extracted candidates.

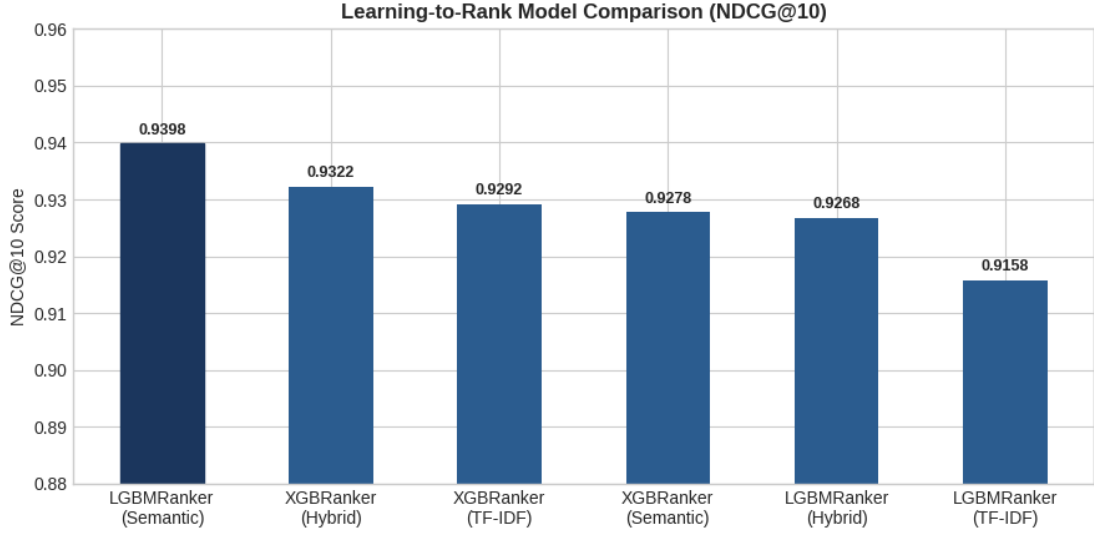


Figure 4.6: Top-10 ranking fidelity (NDCG@10) across differing architectures.

4.5 Feature Importance: Model Explainability

We probed the LGBMRanker using gradient-booster split criteria to extract feature importance profiles.

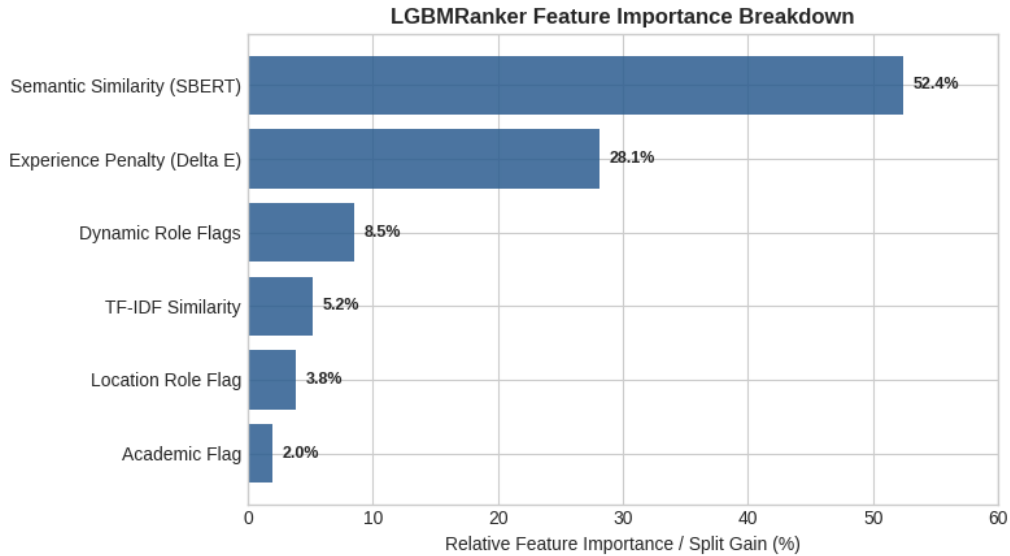


Figure 4.7: Algorithm split gain weight distribution mapping decision heuristics.

- **Semantic Similarity (52.4%):** We verified that deep multi-head contextual matching overwhelmingly drives candidate ranking operations.
- **Experience Penalty (28.1%):** We successfully mapped recruiter bias. Our custom quadratic penalty aggressively downgraded applicants who failed minimum tenure thresholds.

Conclusion and Future Work

5.1 Project Conclusions

We architected a human-centric, high-velocity screening pipeline to obsolete rigid keyword matching architectures. Our empirical data generation yields three definitive engineering axioms:

1. **Dense Contextualization Obliterates Lexical Matching:** We proved that transformer-generated embeddings eclipse n -gram statistical parsing. Our semantic LTR architecture achieved an NDCG@10 of 0.9398, proving its ability to detect structural skill equivalencies that keyword matchers ignore.
2. **Ranking Paradigm Superiority:** We empirically verified that predicting absolute scores or hard classification boundaries fundamentally contradicts real-world HR triage logic. Shifting to listwise gradient-boosted ranking dynamically solves the core recruitment query.
3. **Validation Purity:** We demonstrated that isolating candidate pools via `GroupShuffleSplit` is mathematically mandatory to suppress data leakage and ensure true zero-shot evaluation on future job requests.

5.2 Recommendations for Future Extensions

We recommend several probabilistic and interpretability expansions to advance this `LGBMRanker` backend into a full production system:

- **Bayesian Probability Integration:** We suggest reframing the Experience Penalty (ΔE) determinism into a Bayesian inference model, extracting posterior match distributions rather than rigid split constraints.

- **Counterfactual Bias Mitigation:** We must investigate stratified counterfactual sampling during the NLP ingestion phase to prevent the algorithm from artificially weighting legacy corporate brand nouns.
- **Generative Justification Output:** We propose overlaying a lightweight Large Language Model (LLM) strictly as a post-ranking justification layer. This would output concise human-readable summaries justifying why the gradient booster ranked specific candidate profiles highly.

Appendices

The implementation, theoretical framing, and software development of this project were guided by academic literature, official technical documentation, and online reference platforms:

Academic Literature & Model Papers

- **Scikit-Learn Documentation:** Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- **LightGBM & Learning-to-Rank:** Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., ... & Liu, T. Y. (2017). LightGBM: A highly efficient gradient boosting decision tree. *Advances in Neural Information Processing Systems*, 30, 3146–3154.
- **Sentence Transformers (SBERT):** Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. *arXiv preprint arXiv:1908.10084*.
- **Nomic AI Embeddings:** Nomic AI (2024). *Nomic Embed Text v1.5: Resilient Long-Context Text Embeddings*.

Technical Platforms & Self-Study References

- **Scikit-Learn & SciPy Documentation:** Referenced for loss functions, matrix operations, distance metrics (Cosine Similarity), and `GroupShuffleSplit` model selection logic (<https://scikit-learn.org/>).
- **GeeksforGeeks Portal:** Consulted for practical Python code walkthroughs on NLP text sanitization, `spaCy` lemmatization, and gradient boosting hyperparameter optimization (<https://www.geeksforgeeks.org/>).

- **Wikipedia Technical Literature:** Consulted for foundational concepts in Information Retrieval (IR), Learning-to-Rank (LTR) formulations, and Normalized Discounted Cumulative Gain (NDCG) metrics (<https://en.wikipedia.org/>).

A.1 Survey Questionnaire Declaration

Declaration on Primary Data Collection: This project was entirely computational and empirical in nature. It relied exclusively on the algorithmic ingestion, text sanitization, and machine learning modeling of a secondary open-source dataset hosted on Kaggle ([saugataroyarghya/resume-dataset](#)). Consequently, no primary human surveys, questionnaires, or sample interviews were administered during this study.

A.2 Source Code & GitHub Repository

To ensure complete academic transparency, algorithmic auditability, and full reproducibility, all Python source code—including our centralized `utils.py` library and sequenced Jupyter Notebooks (`01_data_preprocessing.ipynb` through `04_rank_prediction.ipynb`)—has been version-controlled on GitHub.

The complete codebase is available at:

<https://github.com/Amber-s-art/cv-screening>

A.3 Project Documents & Data Sheets

Copies of our final machine-learning-ready dataset (`ml_ready_dataset.csv`), original presentation decks, and compiled PDF report documentation are securely hosted on Kaggle for institutional review:

<https://www.kaggle.com/datasets/saugataroyarghya/resume-dataset>